

测试驱动的开发

大纲

- 什么是测试驱动的开发
- Design for test：如何设计可测试的软件

测试驱动的开发与自动测试技术

- 测试驱动开发可以达到自动测试的效果，但本质上测试驱动的开发并不是自动测试技术。
- 测试驱动的开发本质上可以被看成是一种设计的技术。

什么是测试驱动的开发

- 先写单元测试用例，再写代码；
- 由测试来决定需要什么样的代码；
- 由程序员编写和维护完整的测试用例集；
- 仅当代码有了相应的测试代码，该代码才能作为成品代码；

一个简单的演示

编写一个能够执行加法和减法的计算器

改变习惯

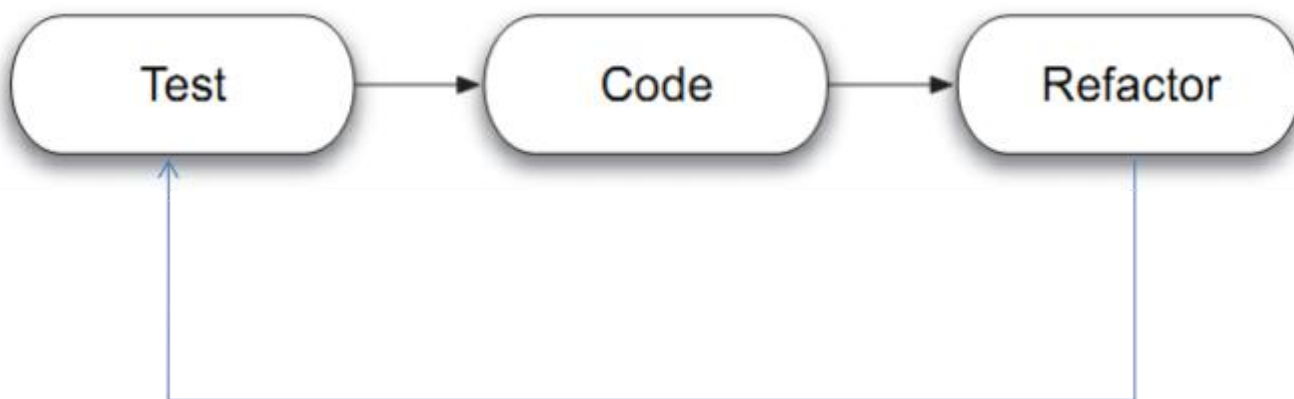
- 通过写测试代码整理设计思路
- 不必害怕错误
 - 让计算机帮你记住
 - 需要实现哪些类、接口和方法
 - 哪些方法还没有实现或者还存在问题

测试驱动的开发的优点

- 能够保证编写单元测试
- 使得程序员获得满足感，从而始终如一地坚持编写测试
- 有助于澄清接口和行为的细节
- 可证明、可再现、可自动验证
- 改变事物的信心

开发过程的变化

- 采用测试驱动的开发后：

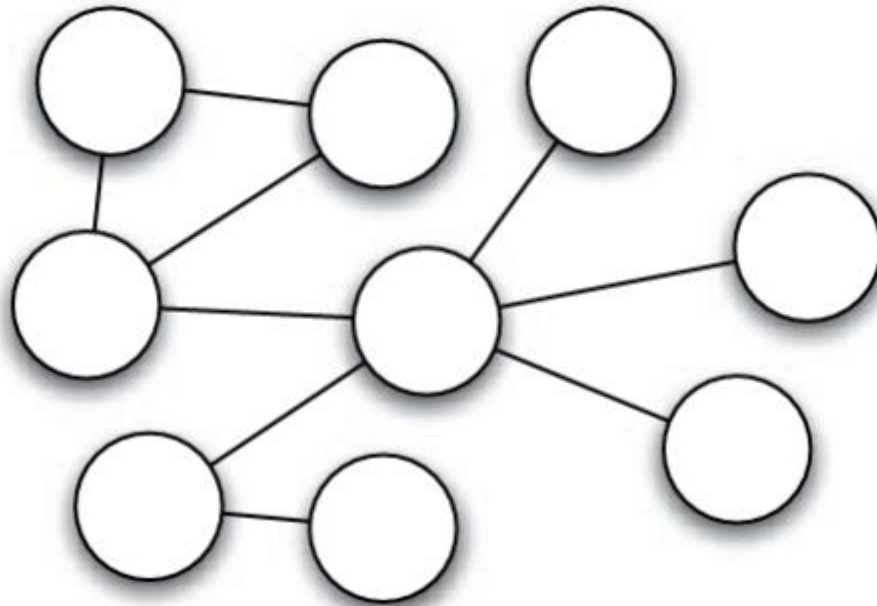


引入测试驱动的开发

- 先写测试用例的习惯不容易形成
- 即便是补写测试用例，有些代码也难以测试。
 - 以掷骰子游戏为例

Designing for testing

- To test a component, you must be able to **control its input** (and internal state) and observe its output



Design for testing

- Separation of interface and implementation
 - Allows substitution of implementation to tests
- Dependency injection

Active acquisition of dependency



dependency injection



Design for testing - Mock Objects

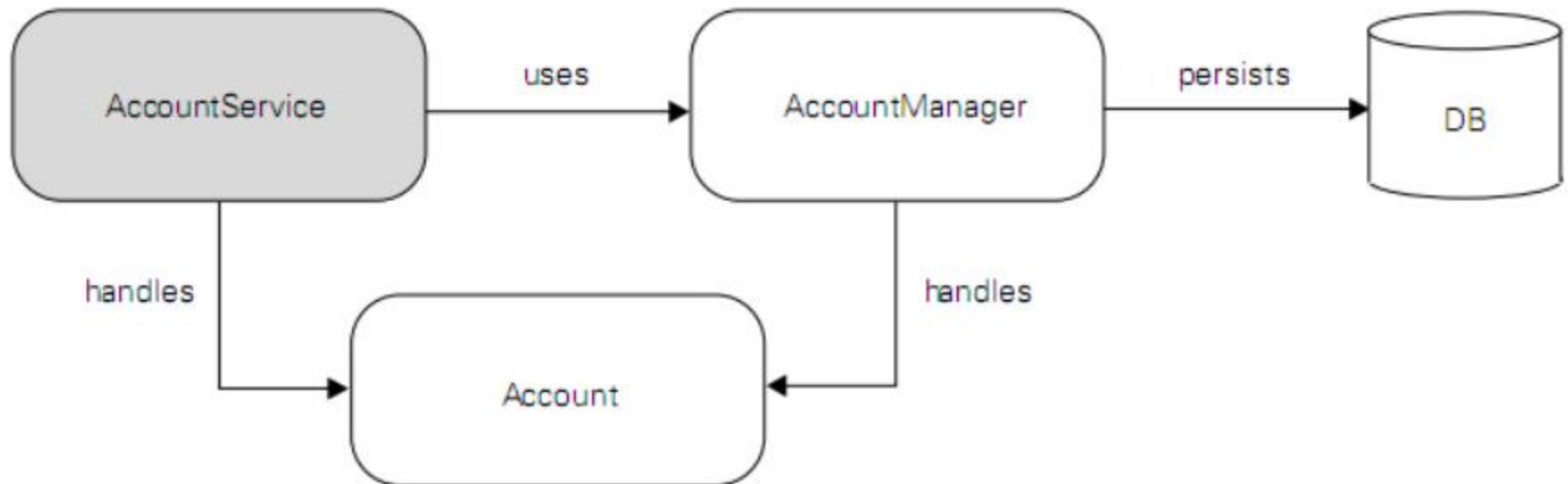
- When your implementation requires a resource which is unavailable for testing
 - External system or database is simulated.
 - Another use of Factory, the mock implementation stubs out and returns the anticipated results from a request.

Introducing mock objects

- Definition
 - A mock object is an object created to stand in for an object that your code will be collaborating with. Your code can call methods on the mock object, which will deliver results as set up by your tests.
- Mock objects are perfectly suited for testing a portion of code logic in isolation from the rest of the code.

Mock tasting: a simple example

- Make a bank transfer from one account to another



Class under test

```
public class AccountService
{
    private AccountManager accountManager;

    public void setAccountManager(AccountManager manager)
    {
        this.accountManager = manager;
    }

    public void transfer(String senderId, String beneficiaryId,
        long amount)
    {
        Account sender =
            this.accountManager.findAccountForUser(senderId);
        Account beneficiary =
            this.accountManager.findAccountForUser(beneficiaryId);

        sender.debit(amount);
        beneficiary.credit(amount);

        this.accountManager.updateAccount(sender);
        this.accountManager.updateAccount(beneficiary);
    }
}
```



AccountManager as Interface

```
package junitbook.fine.tasting;  
  
public interface AccountManager  
{  
    Account findAccountForUser(String userId) ;  
    void updateAccount(Account account) ;  
}
```

Test Case for AccountService

```
public class TestAccountService extends TestCase
{
    public void testTransferOk()
    {

        Account senderAccount = new Account("1", 200);
        Account beneficiaryAccount = new Account("2", 100);

        AccountService accountService = new AccountService();

        accountService.transfer("1", "2", 50);

        assertEquals(150, senderAccount.getBalance());
        assertEquals(150, beneficiaryAccount.getBalance());
    }
}
}
```

1

2

3



MockAccountManager

```
public class MockAccountManager implements AccountManager
{
    private Hashtable accounts = new Hashtable();

    public void addAccount(String userId, Account account)
    {
        this.accounts.put(userId, account);
    }

    public Account findAccountForUser(String userId)
    {
        return (Account) this.accounts.get(userId);
    }

    public void updateAccount(Account account)
    {
        // do nothing
    }
}
```



Test Case for AccountService

```
public class TestAccountService extends TestCase
{
    public void testTransferOk()
    {
        MockAccountManager mockAccountManager =
            new MockAccountManager();
        Account senderAccount = new Account("1", 200);
        Account beneficiaryAccount = new Account("2", 100);

        mockAccountManager.addAccount("1", senderAccount);
        mockAccountManager.addAccount("2", beneficiaryAccount);

        AccountService accountService = new AccountService();
        accountService.setAccountManager(mockAccountManager);

        accountService.transfer("1", "2", 50);

        assertEquals(150, senderAccount.getBalance());
        assertEquals(150, beneficiaryAccount.getBalance());
    }
}
}
```

1

2

3

Using mock objects as a refactoring technique

- What's wrong with the following code:

```
public class DefaultAccountManager implements AccountManager
{
    private static final Log LOGGER =
        LoggerFactory.getLog(AccountManager.class);

    public Account findAccountForUser(String userId)
    {
        LOGGER.debug("Getting account for user [" + userId + "]");
        ResourceBundle bundle =
            PropertyResourceBundle.getBundle("technical");
        String sql = bundle.getString("FIND_ACCOUNT_FOR_USER");

        // Some code logic to load a user account using JDBC
        [...]
    }
    [...]
}
```

1

2



Using mock objects as a refactoring technique

- Easy refactoring

```
public Account findAccountForUser(String userId)
{
    this.logger.debug("Getting account for user ["
        + userId + "]");
    this.configuration.getSQL("FIND_ACCOUNT_FOR_USER");
    // Some code logic to load a user account using JDBC
    [...]
}
```

Using mock objects as a refactoring technique

- Inversion of Control(IoC)
 - Applying the IoC pattern to a class means removing the creation of all object instances for which this class is not directly responsible and

```
public void testFindAccountByUser()
{
    MockLog logger = new MockLog();           ❶
    MockConfiguration configuration = new MockConfiguration();
    configuration.setSQL("SELECT * [...]");    ❷

    DefaultAccountManager am =
        new DefaultAccountManager(logger, configuration);  ❸

    Account account = am.findAccountForUser("1234");

    // Perform asserts here
    [...]
}
```



什么是JUnit

- Junit是一个测试框架，它的目标是简化单元测试的开发、运行和报告，主要包含以下的功能：
 - 断言(assertions)
 - 测试类和测试集(suites)
 - 测试运行
 - 测试结果报告
- 对于绝大多数的程序设计语言，都有类似的元体测试框架，他们统称为xUnit。

TestRunners

- 基于文本
 - 轻量级，快速
 - 从命令行启动

java StringTest

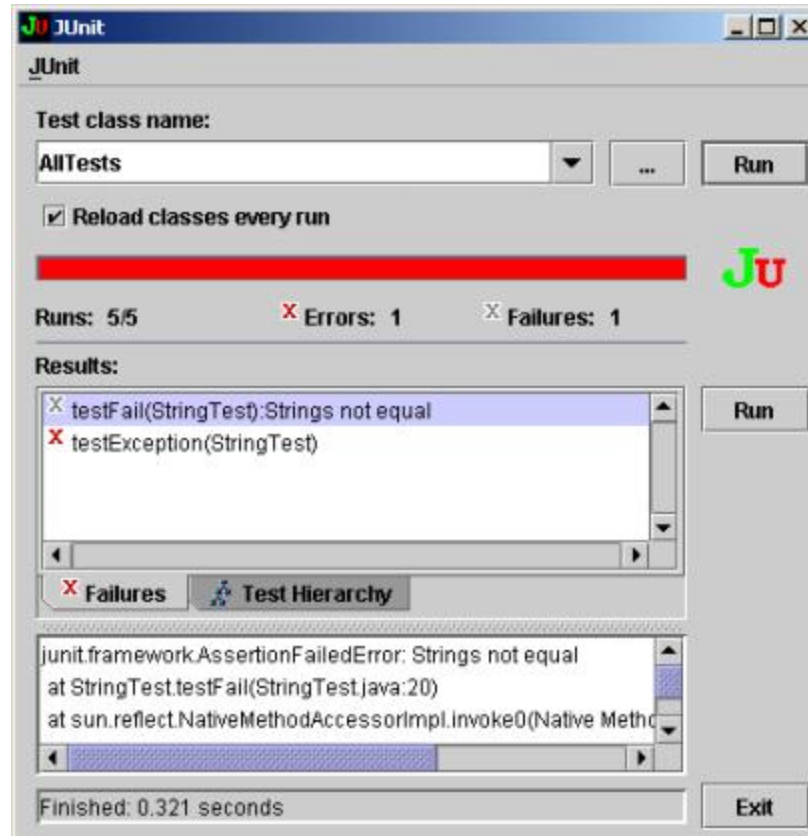
.....

Time: 0.05

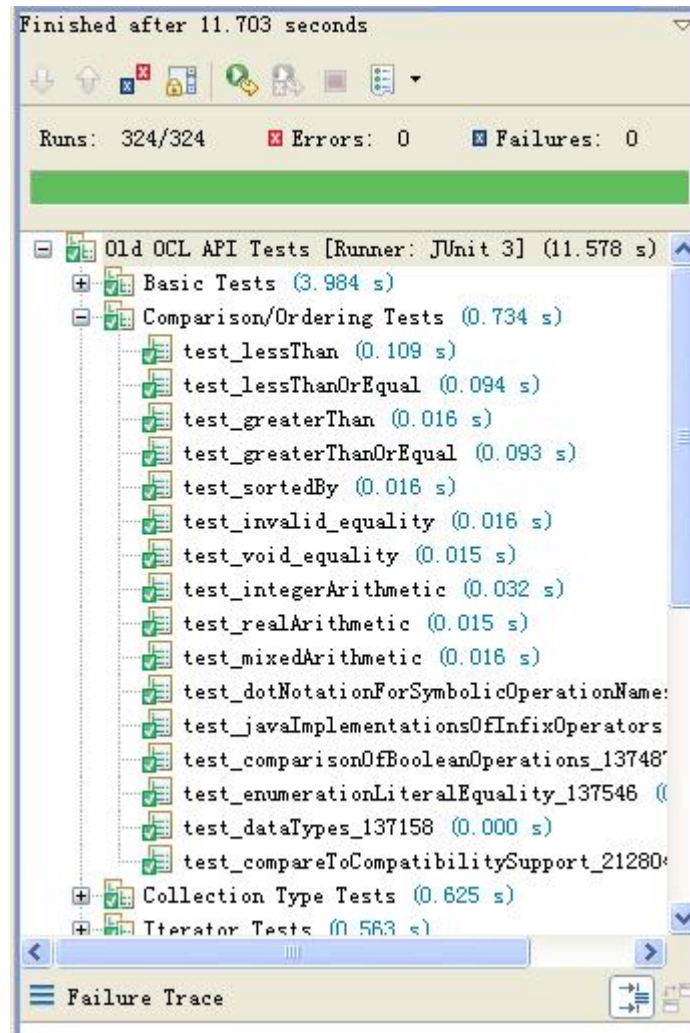
Tests run: 7, Failures: 0, Errors: 0

TestRunners - Swing

- 图形界面，通过启动
`java junit.swingui.TestRunner`



TestRunner : Eclipse IDE



vscode

The image shows the Visual Studio Code (VS Code) interface with a Java test file open and its results displayed.

Editor View:

- File Explorer: `junitdemo > src > test > java > junitdemo > CalculatorTest.java`
- Code Editor:

```
1 package junitdemo;
2
3 import static org.junit.Assert.assertEquals;
4
5 import org.junit.Test;
6
7 public class CalculatorTest {
8     @Test
9     public void testAdd(){
10         Calculator calc = new Calculator();
11         int result = calc.add(1,2);
12         assertEquals(3, result);
13     }
14 }
15
```

Java Test Report View:

- Test Results Summary:
 - All 1
 - Passed 1
- Test Details:
 - Test Name: `junitdemo.CalculatorTest`
 - Test Method: `> testAdd`
 - Status: **Passed**
 - Duration: 0s

A speaker icon is visible in the bottom right corner of the interface.

与自动构造工具集成： Maven

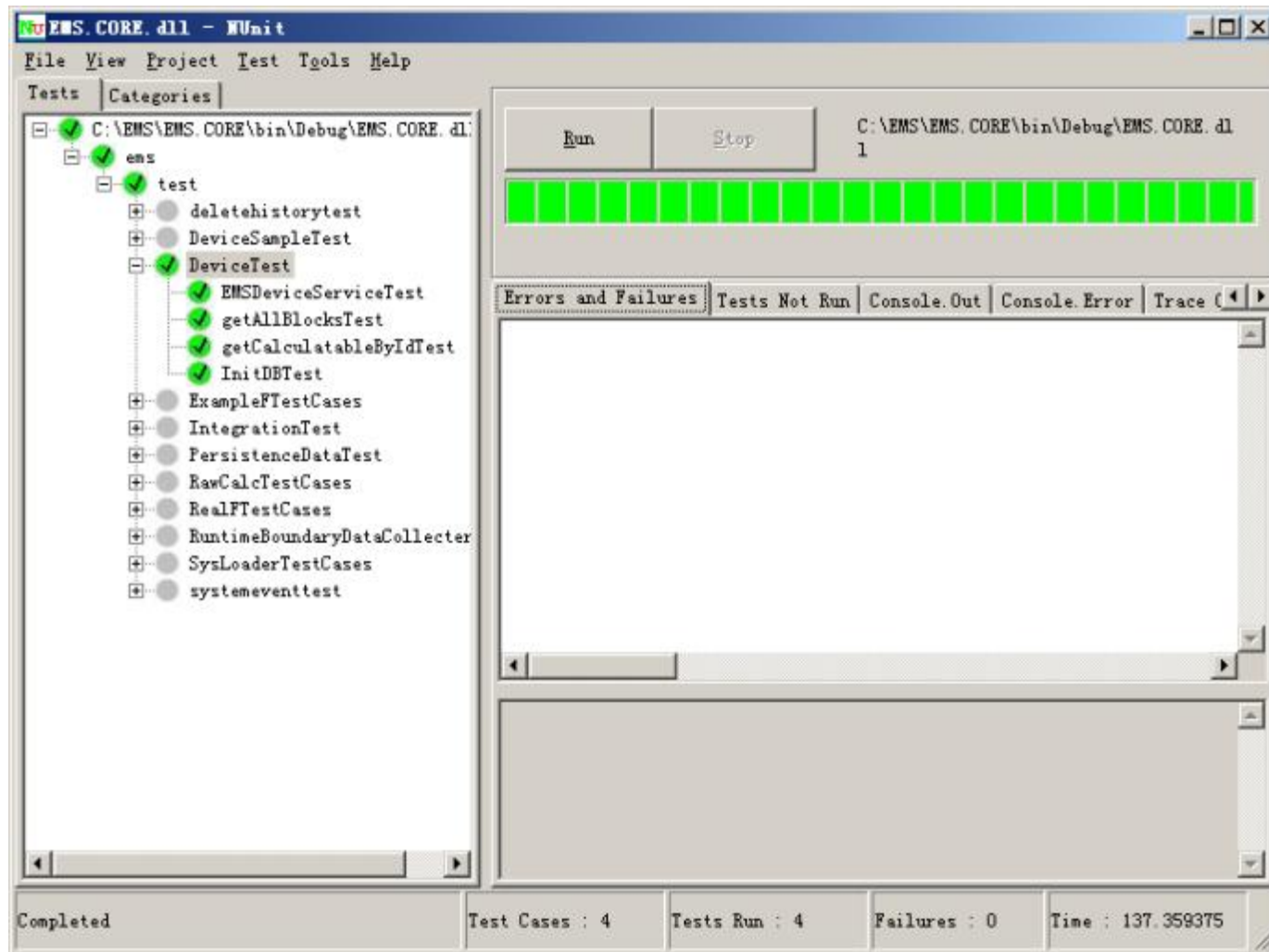
```
mvn install
```

```
[INFO]
[INFO] -----
[INFO] Building demo 1.0.0
[INFO] -----
.....
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running demo.AllTests
[INFO] Tests run: 33, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 13.252 sec

[INFO] Results :

[INFO] Tests run: 33, Failures: 0, Errors: 0, Skipped: 0
.....
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ demo ---
[INFO] Building jar:demo-1.0.0.jar
[INFO]
.....
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 22.871 s
[INFO] Finished at: 2020-04-04T09:10:38+08:00
[INFO] Final Memory: 20M/334M
[INFO] -----
```

TestRunnit: .NET平台下的NUnit



编写测试用例时的几点注意事项

- 确保测试没有副作用，也就是说测试用例运行前后系统的状态保持不变-
 - 确保测试用例间没有依赖关系，每一个测试用例都能够独立运行
- 测试用例也是程序，需要仔细地设计，比如恰当使用继承、组合关系，提取公共函数等。
- 能快速运行。如果测试运行需要很长时间，会阻碍开发人员经常运行

Demo

掷骰子游戏的解决方案

Test each time you touch code

- Next bug you fix, first write a test that *would have* caught it.
 - Impt. to write the test first. First test should fail.
- If the bit of code isn't testable (in-line), consider refactoring into a subroutine.
- Start a library file with all of your subroutines.